Verification Report: eng2sva Project

Overview

In this project, I built a small tool called `eng2sva` to see if I could write hardware specifications in plain, structured English and have them automatically checked using formal verification tools (specifically Yosys and SymbiYosys).

To test whether the setup actually works in practice, I implemented six standard digital designs in Verilog. For each design, I wrote two versions:

1. A **clean reference version** that follows the intended hardware behavior.
2. A **buggy version** where I introduced a single, realistic logic bug.

I then wrote requirements in English, translated them into SystemVerilog Assertions (and synthesizable formal monitors), and ran formal proofs on both versions of every design.

Overall Results (Kill Table)

Here is the high-level summary of my test run across all six designs:

Design	Injected Bug	Did Formal Catch It?	Properties That Flagged It
Arbiter	Priority logic was removed; both requesters could get a grant at the same time.	**Yes**	`A01`, `A05`
Counter	Wraps around to 0 on overflow instead of holding at 15.	**Yes**	`C03`
FIFO	Keeps accepting writes when full, overflowing memory.	**Yes**	`F02`, `F05`, `F06`
PWM Generator	Used `<=` instead of `<`, turning output ON when duty is 0.	**Yes**	`P01`, `P02`, `P03`
Shift Register	Shift enable mistakenly prioritized over parallel load.	**Yes**	`S02`, `S03`

Traffic Light	Green light skips yellow and transitions directly to red.	**Yes**	`T05`, `T06`

Every single injected bug was caught by at least one written requirement, while the clean designs passed all checks without false alarms.

What Happened on Each Design

1. Arbiter (`designs/rtl/arbiter.v`)

- **What it does**: A 2-port fixed-priority arbiter where Request 0 always wins over Request 1.
- **The bug**: I replaced the priority check with direct assignments (`gnt0 <= req0; gnt1 <= req1;`), allowing both grants to be active at once.
- **Outcome**: Property `A01` (*"grant 0 and grant 1 are mutually exclusive"*) and `A05` (*"If request 0 and request 1 then grant 1 equals 0 in the next cycle"*) immediately failed on the buggy version within a fraction of a second, while both passed with full mathematical induction on the clean design.

2. Saturating Counter (`designs/rtl/counter.v`)

- **What it does**: A 4-bit up-counter that counts from 0 up to 15 when enabled, and stays pegged at 15 once full.
- **The bug**: The counter increments unconditionally on enable, causing it to roll over from 15 to 0.
- **Outcome**: Property `C03` (*"If count equals 15 and en then count equals 15 in the next cycle"*) held on the correct RTL but failed on the buggy RTL at cycle 19. SBY produced a trace showing the counter jumping from 15 directly to 0.

3. Synchronous FIFO (`designs/rtl/fifo.v`)

- **What it does**: A 4-deep, 8-bit FIFO that ignores write attempts when completely full.
- **The bug**: I forced `wr_valid = wr_en`, bypassing the full check and writing into occupied slots.
- **Outcome**: Three separate properties flagged this bug:
  - `F02`: The FIFO count exceeded 4.
  - `F05`: After attempting to write into a full FIFO, the count jumped to 5 rather than staying at 4.

- `F06`: Count did not remain stable during write attempts on a full buffer.

4. PWM Generator (`designs/rtl/pwm.v`)

- **What it does**: Compares a free-running 8-bit counter against a duty cycle value. Output should only be high when `cnt < duty`.
- **The bug**: I changed `<` to `<=`. If someone sets `duty = 0`, the output should always stay off (0% duty cycle). With `<=`, the output pulses high whenever `cnt == 0`.
- **Outcome**: Properties `P01`, `P02`, and `P03` caught this right away. Under formal evaluation, driving `duty = 0` produced an unexpected high output on step 0 of the buggy module.

5. Shift Register with Load (`designs/rtl/shift_reg.v`)

- **What it does**: 8-bit shift register where `load` is supposed to take precedence over `shift_en`.
- **The bug**: I swapped the `if/else` order so `shift_en` took priority over `load`.
- **Outcome**: Property `S03` (*"If load and shift_en then q equals din in the next cycle"*) failed on the buggy version because the register shifted bits instead of latching `din`.

6. Traffic Light Controller (`designs/rtl/traffic.v`)

- **What it does**: Cycles states in order: `RED -> GREEN -> YELLOW -> RED`.
- **The bug**: The GREEN state transition was hardcoded to jump straight to RED, skipping YELLOW entirely.
- **Outcome**: Property `T05` (*"state equals GREEN must never be followed by state equals RED in the next cycle"*) failed on the buggy design in the very first state transition. Property `T06` (*"within 4 cycles"*) also timed out without ever seeing yellow.

Translation & Error Handling Checks

To make sure the translator does not guess or do something unexpected when given invalid input, I included deliberate edge cases:

1. **Subjective / Vague English Requirements**:
   - Sentence: *"The counter should feel responsive and fast."* (Property `C07`)
   - Sentence: *"The lights must look pleasant to motorists."* (Property `T07`)

- **Result**: The parser cleanly returned `UNTRANSLATABLE`. It did not fabricate dummy logic or guess what "fast" or "pleasant" meant.

2. **Typo in Signal Name**:
   - Sentence: *"fifo_cnt must never exceed 4"* (Property `F07`, where the actual dictionary term is `fifo count` or `cnt`).
   - **Result**: The parser halted with an `UNKNOWN PHRASE` error and printed the closest matching dictionary candidates: `['fifo count', 'the fifo count', 'cnt']`.

Practical Takeaways

1. **Free inputs need clear thinking**: Because formal tools drive inputs with random values, any combinational output (like in the PWM) can change whenever an input changes. Writing assertions made me think carefully about what is actually guaranteed versus what relies on external inputs.
2. **Translation is fast and predictable**: Using a structured regex-to-template approach meant translation took less than 0.05 seconds with zero hallucination risk.
3. **Open-source formal is ready for small RTL**: With Yosys and SBY, running 40+ formal proofs across all 6 modules took under 30 seconds total on a regular machine.